

Lab 3

[valid 2024-2025]

Interfaces, Collections, Generics

In busy airports, flights must be scheduled carefully to avoid conflicts on shared resources, like runways.

Consider an *airport* that has a specific number of *runways*. The airport is used by various *aircraft*, such as *airliners*, *freighters*, *drones*, etc., each having common properties, such as model, tail number or name (call sign), and specific properties, such as the the wing span, the number of passengers it can carry, the maximum payload, or the battery life.

Every day, there are a number of *flights* which arrive at the airport. Each flight is operated by an aircraft, has an identification number and must land at the airport in a specific interval.

Two flights are considered to be in conflict if their landing intervals intersect. The goal is to assign each flight a runway, such that conflicting flights are scheduled on different runways.

The main specifications of the application are:

Compulsory (1p)

- Create an object-oriented model of the problem. You should have at least the following classes `Aircraft`, `Airliner`, `Freighter`, `Drone`. The natural ordering of their objects is given by their names.
 - Create the interfaces `PassengerCapable` and `CargoCapable`. The classes above must implement these interfaces accordingly.
 - Create `Aircraft` objects and an array containing all aircraft that can transport goods.
-

Homework (2p)

- Implement the `Airport` and the `Flight` classes. The airport will contain a *list* of runways. Each flight will have its landing interval specified using a pair of `LocalTime` objects.
 - Create a class that describes the scheduling problem. An instance of this problem will contain an airport and a *set* of flights.
 - Implement a simple algorithm in order to solve the problem. The solution must *map* each flight to a runway.
-

Bonus (2p)

- The airport manager wants to ensure that the runways are used in an equitable manner. Implement an algorithm that creates a schedule in which the number of flights assigned to each runway is either equal or differs by at most one.
If it is not possible, the algorithm should determine how many runways would be necessary in order to satisfy the requirement.
Alternatively, you may consider modifying the landing intervals of the planes, keeping them more in the air, in order to use the existing runways equitably.
- Generate random instances of the problem and test your algorithm.

Resources

- [Slides](#)
- [The Java Tutorials: Interfaces](#)
- [The Java Tutorials: Collections](#) ([Know Thy Complexities!](#))

- [The Java Tutorials: Generics](#)
- [The Date-Time package](#)

Objectives

- Create interfaces to describe specifications.
- Create multiple implementations of an interface.
- Understand the differences between abstract classes and interfaces.
- Use packages to organize the classes and interfaces of the application.
- Use collections and generics.
- Understand the space-time tradeoff of various types of collections.